

Coffee Shop Kiosk Development Guide

<https://gemini.google.com/app/5aba76fa2581a794>

User prompt: im want to make a html,css, and js but without relying on ai but give me some guides and also assist me on making this according to this attached files

Response: Here is a complete HTML and CSS structure for your interactive kiosk, designed around a relaxed, acoustic coffee shop vibe called "Ceffie Break."

As required by your assignment's AI policy, I have only generated the HTML and CSS. All JavaScript must be written by you without AI assistance.

Below the code, I've provided a blueprint to help you write your JavaScript file and meet all the grading criteria.

1. HTML (index.html)

This structure is ready for the 5 events you need to implement. It includes buttons, a form, text inputs, and hidden elements to target.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ceffie Break - Interactive Kiosk</title>
  <link rel="stylesheet" href="style.css">
</head>
<body class="light-mode">
  <header>
    <h1>Ceffie Break</h1>
    <p>An acoustic, relaxed social experience</p>
    <button id="theme-toggle">Toggle Dark Mode</button>
  </header>

  <main>
    <!-- Section for the self-removing event -->
    <section id="special-offer">
      <div class="offer-box">
        <h2>☕ Today's Special!</h2>
        <p>Get 20% off your first acoustic session brew.</p>
        <button id="claim-offer">Claim Offer</button>
      </div>
    </section>

    <!-- Section for input and mouseover events -->
    <section class="menu-section">
      <h2>Our Menu</h2>
      <input type="text" id="menu-search" placeholder="Search for a drink...">
      <p id="search-feedback"></p>

      <div class="menu-grid">
        <div class="menu-card" id="card-1">
          <h3>Classic Espresso</h3>
          <p class="hidden-info">Rich, bold, and perfect for conversation.</p>
        </div>
        <div class="menu-card" id="card-2">
          <h3>Vanilla Latte</h3>
          <p class="hidden-info">Smooth and sweet, just like acoustic tunes.</p>
        </div>
      </div>
    </section>

    <!-- Section for form submit and keyup events -->
    <section class="feedback-section">
      <h2>Share Your Thoughts</h2>
```

```

    <form id="feedback-form">
      <label for="feedback-text">Message:</label>
      <textarea id="feedback-text" rows="3" placeholder="Tell us about your vibe..."></textarea>
      <p>Characters typed: <span id="char-count">0</span></p>
      <button type="submit">Submit Feedback</button>
    </form>
    <p id="form-success-msg" class="hidden">Thanks for sharing your thoughts!</p>
  </section>
</main>

<!-- Make sure to link your JS file here -->
<script src="script.js"></script>
</body>
</html>

```

2. CSS (style.css)

This styling uses a clean, minimalist layout and incorporates the Bookman Old Style font for the headers.

```

:root {
  --bg-color: #fdfaf6;
  --text-color: #3e2723;
  --primary: #8d6e63;
  --card-bg: #ffffff;
}

body.dark-mode {
  --bg-color: #3e2723;
  --text-color: #fdfaf6;
  --primary: #d7ccc8;
  --card-bg: #4e342e;
}

body {
  font-family: sans-serif;
  background-color: var(--bg-color);
  color: var(--text-color);
  margin: 0;
  padding: 0;
  transition: background-color 0.3s, color 0.3s;
}

header {
  text-align: center;
  padding: 2rem;
  background-color: var(--primary);
  color: white;
  font-family: 'Bookman Old Style', serif;
}

main {
  max-width: 800px;
  margin: 2rem auto;
  padding: 0 1rem;
}

.offer-box {
  background-color: var(--card-bg);
  border-left: 5px solid var(--primary);
  padding: 1rem;
  margin-bottom: 2rem;
  box-shadow: 0 4px 6px rgba(0,0,0,0.1);
}

.menu-grid {

```

```

    display: flex;
    gap: 1rem;
    margin-top: 1rem;
  }

  .menu-card {
    background-color: var(--card-bg);
    padding: 1rem;
    border-radius: 8px;
    flex: 1;
    box-shadow: 0 2px 4px rgba(0,0,0,0.1);
  }

  .hidden-info {
    opacity: 0;
    transition: opacity 0.3s;
  }

  .hidden {
    display: none;
  }

  input, textarea {
    width: 100%;
    padding: 0.5rem;
    margin-top: 0.5rem;
    box-sizing: border-box;
  }

  button {
    background-color: var(--primary);
    color: white;
    border: none;
    padding: 0.5rem 1rem;
    cursor: pointer;
    border-radius: 4px;
    margin-top: 0.5rem;
  }

  button:hover {
    opacity: 0.9;
  }

```

Your JavaScript Guide (script.js)

You need to wire up the interactions using JavaScript yourself. According to your rubric, your script needs at least 5 distinct events attached using `addEventListener()`. You also need to make sure every handler has a one-line comment explaining what triggers it and what it does.

Here is a roadmap of the events you can build to bring this HTML to life:

1. The `click` Event (Theme Toggle):

- **Goal:** Make the `#theme-toggle` button switch the page to dark mode.
- **How:** Use `document.querySelector` to grab the button and the `body`. Attach a listener to the button that toggles the `dark-mode` class on the body.

2. The `mouseover` / `mouseout` Events (Menu Cards):

- **Goal:** Reveal the `.hidden-info` text inside the menu cards when hovering.
- **How:** Select the cards. When a `mouseover` event fires, change the `style.opacity` of the hidden info paragraph to `"1"`. Change it back to `"0"` on `mouseout`.

3. The `input` Event (Search Filter):

- **Goal:** Provide live feedback as the user types in the search bar.
- **How:** Listen for "input" on the #menu-search field. Remember that the event object contains the current value, which you can access and display inside the #search-feedback paragraph.

4. The `keyup` Event (Character Counter):

- **Goal:** Update the character count in real-time.
- **How:** Listen to #feedback-text. Inside the handler, check the `length` of the textarea's value and update the #char-count span's text content.

5. The `submit` Event (Form Submission):

- **Goal:** Show a success message without reloading the page.
- **How:** Listen to "submit" on #feedback-form. **Crucial step:** You must call `preventDefault()` on the event object to stop the browser's default behavior, which is reloading the page. Then, remove the "hidden" class from #form-success-msg.

6. The `removeEventListener` Requirement (Claim Offer):

- **Goal:** Allow the user to click the "Claim Offer" button only once.
- **How:** Create a named function for this handler (not an anonymous arrow function). Inside the function, change the button's text to "Offer Claimed!", and then immediately call `removeEventListener()` to detach that exact listener so future clicks do nothing.

Important Reminders for your Submission: Before you turn in your assignment, test it thoroughly (including edge cases like rapid clicking). Because you used AI to help generate the HTML and CSS, you **must** attach this full chat history (exported or screenshotted) with your final submission to avoid a penalty.
